

1 = NES-VMC 算法在 NETKET 官方 API 中的复现

1.1 = 1. 研究目的

目标：完全基于 NetKet 框架的高层 API (MCState、VMC 驱动) 复现 **NES-VMC (Natural Excited State Variational Monte Carlo) 算法**，用于计算量子多体系统（如 H_2 分子）的前 K 个激发态能量。

要求：

- 使用 NetKet 内置的扩展希尔伯特空间 $h_i \times K$
- 使用 MCState 管理变分态与采样器
- 使用 VMC 驱动自动执行训练循环
- 最终通过训练得到的模型，对角化平均局域能量矩阵，获得基态与激发态能量

1.2 = 2. NES-VMC 算法核心思想

1.2.1 = 2.1 问题背景

在量子力学中，我们通常要求解哈密顿算符 $\hat{H}$ 的本征值问题，即找到最低的 K 个本征函数。对于量子多体系统，直接对角化哈密顿矩阵通常是不可行的，因为希尔伯特空间的维度随粒子数指数增长。

NES-VMC 将原系统前 K 个激发态的求解问题**等价转化为一个"扩展系统"的基态求解问题**。

1.2.2 = 2.2 扩展希尔伯特空间

设 $x = (x_1, \dots, x_N)$ 表示一组包含 N 个粒子的粒子集 (particle set)，其中 x_i 表示第 i 个粒子的状态。扩展希尔伯特空间由 K 个原系统副本张量积构成，每个配置对应 K 个组态 $\mathbf{x} = (x^1, \dots, x^K)$ 。

1.2.3 = 2.3 TotalAnsatz 的构成

设 ψ_i 表示第 i 个 N 粒子波函数（可能未归一化），则 **TotalAnsatz** 定义为矩阵 $\Psi(\mathbf{x}) \in \mathbb{R}^{K \times K}$ 的行列式：

$$\Psi(\mathbf{x}) \equiv \det \begin{pmatrix} \psi_1(x^1) & \psi_2(x^1) & \cdots & \psi_K(x^1) \\ \psi_1(x^2) & \psi_2(x^2) & \cdots & \psi_K(x^2) \\ \vdots & \vdots & \ddots & \vdots \\ \psi_1(x^K) & \psi_2(x^K) & \cdots & \psi_K(x^K) \end{pmatrix}$$

其中：

- $\Psi(\mathbf{x}) \in \mathbb{R}^{K \times K}$ ：将所有电子集合与所有波函数结合的矩阵
- $\psi_i(x^j)$ ：第 i 个单态 Ansatz 在第 j 个粒子集上的值
- $\Psi(\mathbf{x}) = \det(\Psi(\mathbf{x}))$ ：总 Ansatz，可以看作是由 N 粒子波函数组成的未归一化 Slater 行列式

关键性质：通过将总 Ansatz 表示为单态 Ansatz 的行列式，可以防止不同 Ansatz 坍缩到同一状态，而不需要显式要求它们正交。

1.2.4 = 2.4 扩展哈密顿量

定义扩展哈密顿量 $\tilde{H} = \hat{H}_1 \oplus \hat{H}_2 \oplus \cdots \oplus \hat{H}_K$ ，其中 $\hat{H}_i$ 是仅作用于第 i 个粒子集的哈密顿量。 $\tilde{H}$ 的基态能量等于原系统 $\hat{H}$ 最低 K 个能量之和，其基态波函数正是上述行列式形式的 Ψ^* 。

1.3 = 3. 损失函数

1.3.1 = 3.1 目标函数（Rayleigh 商）

NES-VMC 的目标函数为扩展哈密顿量关于总 Ansatz 的 Rayleigh 商：

$$\mathcal{L} = \frac{\langle \Psi | \tilde{H} | \Psi \rangle}{\langle \Psi | \Psi \rangle}$$

利用矩阵行列式引理，可以将其重写为迹形式：

$$\mathcal{L} = \frac{\langle \Psi | \tilde{H} | \Psi \rangle}{\det(S)} = \mathrm{Tr}(S^{-1} \tilde{H})$$

其中 S 为重叠矩阵：

$$S = \begin{pmatrix} \langle \psi_1 | \psi_1 \rangle & \cdots & \langle \psi_1 | \psi_K \rangle \\ \vdots & \ddots & \vdots \\ \langle \psi_K | \psi_1 \rangle & \cdots & \langle \psi_K | \psi_K \rangle \end{pmatrix}$$

1.3.2 = 3.2 局域能量矩阵

通过 Monte Carlo 采样，损失函数可以写成期望值形式：

$$\mathcal{L} = \mathbb{E}_{\Psi^2}[\tilde{H} \Psi]$$

定义**局域能量矩阵**为：

$$E_L(\mathbf{x}) \equiv \Psi^{-1}(\mathbf{x}) \tilde{H} \Psi(\mathbf{x})$$

这是一个 $K \times K$ 矩阵，其迹即为标量局域能量。当 $K = 1$ 时，这退化为标准 VMC 中的局域能量。

简化形式：

$$E_L(\mathbf{X}) = \mathrm{Tr}(M^{-1}(\mathbf{X}), H_M(\mathbf{X}))$$

其中 M 为行列式矩阵， H_M 为哈密顿量作用在每一列上的矩阵。

1.4 = 4. 梯度公式

1.4.1 = 4.1 标准 VMC 梯度回顾

对于基态 VMC，能量关于变分参数 θ 的梯度为：

$$\nabla_{\theta} \frac{\langle \psi | \hat{H} | \psi \rangle}{\langle \psi | \psi \rangle} = 2 \mathbb{E} \{ x \sim \psi^2 \} \left[\left(E_L(x) - \mathbb{E} \{ x' \sim \psi^2 \} [E_L(x')] \right) \nabla_{\theta} \log |\psi(x)| \right]$$

1.4.2 = 4.2 NES-VMC 梯度

对于总 Ansatz，梯度计算类似。定义对数幅度：

$$\log |\Psi(\mathbf{x})| = \log \det(\Psi(\mathbf{x})) = \mathrm{Tr}(\log(\Psi(\mathbf{x})))$$

梯度公式为：

$$\nabla_{\theta} \mathcal{L} = 2 \mathbb{E} \{ \mathbf{x} \sim \Psi^2 \} \left[\left(E_L(\mathbf{x}) - \bar{E}_L \right) \nabla_{\theta} \log |\Psi(\mathbf{x})| \right]$$

其中 $\bar{E}_L = \mathbb{E} \{ E_L(\mathbf{x}) \sim \Psi^2 \}$ 是局域能量矩阵的期望值。

1.4.3 = 4.3 批量 walker 的梯度估计

与标准 VMC 类似，可以使用同一批次中独立的 walker 来获得无偏梯度估计：

$$\nabla_{\theta} \mathcal{L} = \frac{N-1}{2N} \mathbb{E} \{ x_1, \dots, x_N \} \left[\frac{1}{N} \sum_{i=1}^N \left(E_L(x_i) - \frac{1}{N} \sum_{j=1}^N E_L(x_j) \right) \nabla_{\theta} \log |\Psi(x_i)| \right]$$

1.5 = 5. 激发态能量提取

1.5.1 = 5.1 能量矩阵的对角化

训练完成后，通过大量采样累积局域能量矩阵：

$$\bar{E}_L = \mathbb{E} \{ E_L(\mathbf{x}) \sim \Psi^2 \}$$

然后对 $\bar{E}_L$ 进行对角化：

$$\bar{E}_L = U \Lambda U^{-1}$$

其中 $\Lambda = \mathrm{diag}(E_1, E_2, \dots, E_K)$ 包含按能量排序的本征值。

1.5.2 = 5.2 物理解释

当单态 Ansatz 是本征函数的线性组合 $\psi_i = \sum_j a_{ij} \psi_j^*$ 时，有：

$$\hat{H} \Psi = A^{-1} \Lambda A \Psi$$

其中 A 是系数矩阵。因此，通过对角化可以直接获得各激发态的能量 $E_1, E_2, \dots, E_K$ 。

1.6 = 6. 关键特性总结

- **扩展希尔伯特空间**：\$K\$ 个原系统副本的张量积
- **Slater 行列式形式**：自动保证正交性（行列式为零条件）
- **局域能量矩阵**：\$K \times K\$ 矩阵，迹为标量能量
- **能量提取**：对角化平均局域能量矩阵获得各激发态能量

以下是我的代码实现：

```

"""
NES-VMC (Natural Excited State Variational Monte Carlo) 算法实现

本文件实现基于原生 JAX 和部分 NetKet 的 NES-VMC 算法，用于计算量子多体系统的激发态能量。
"""
import jax
import jax.numpy as jnp
import netket as nk
import netket.experimental as nkx
import numpy as np
from pyscf import gto, scf, fci
from flax import linen as nn
import flax.nnx as nnx
import optax
from tqdm import tqdm
from functools import partial
from jax import flatten_util
import orbax.checkpoint as ocp
from pathlib import Path
from jax import jit, vmap, grad, value_and_grad
import jax.numpy as jnp
import jax
from functools import partial

# =====
# 1. 全局参数 & H2 分子定义
# =====
# ===== H2 分子定义 & FCI 基准 =====
bond_length = 1.4
geometry = [('H', (0., 0., 0.)), ('H', (bond_length, 0., 0.))]
mol = gto.M(atom=geometry, basis='STO-3G', verbose=0)
mf = scf.RHF(mol).run(verbose=0)

# FCI 精确基准
cisolver = fci.FCI(mf)
cisolver.nroots = 4
E_fcis, fcivec = cisolver.kernel()
print("="*60)
print("H2 FCI 基准能量")
print("="*60)
for i, e in enumerate(E_fcis):
    exc = (e - E_fcis[0]) * 27.2114
    print(f"E{i} = {e:.8f} Ha | 激发能: {exc:.4f} eV")

# ===== NetKet 哈密顿量和采样器 =====

```

```

ha = nkx.operator.from_pyscf_molecule(mol)

hi = nkx.hilbert.SpinOrbitalFermions(
    n_orbitals=2,
    s=1/2,
    n_fermions_per_spin=(1,1),
)
K=2
hi_ext = hi**K
edges = [(0, 1), (2, 3)]
g = nk.graph.Graph(edges=edges)
single_rule = nk.sampler.rules.FermionHopRule(hi, graph=g)
tensor_rule = nk.sampler.rules.TensorRule(hi_ext, [single_rule] * K)
sampler = nk.sampler.MetropolisSampler(hi_ext, rule=tensor_rule, n_chains=100, sweep_size=32)

class SingleStateAnsatz(nnx.Module):
    """单态 Ansatz: 适配费米子系统的复数值 FFNN"""

    def __init__(self, n_spin_orbitals: int, hidden_dim: int = 16, *, rngs: nnx.Rngs):
        super().__init__()
        self.n_spin_orbitals = n_spin_orbitals
        self.linear1 = nnx.Linear(n_spin_orbitals, hidden_dim, rngs=rngs, param_dtype=complex)
        self.linear2 = nnx.Linear(hidden_dim, hidden_dim, rngs=rngs, param_dtype=complex)
        self.output = nnx.Linear(hidden_dim, 1, rngs=rngs, param_dtype=complex)

    def __call__(self, x: jax.Array) -> jax.Array:
        h = nnx.tanh(self.linear1(x))
        h = nnx.tanh(self.linear2(h))
        out = self.output(h)
        return jnp.squeeze(out)

class NESTotalAnsatz(nnx.Module):
    """
    NES-VMC 总波函数 Ansatz
    支持自动处理:
        - 单样本: 输入 (K, n_spin_orbitals)
        - 批量样本: 输入 (batch_size, K, n_spin_orbitals)
    输出:
        log_psi_total : 标量 或 (batch_size,)
        log_M_matrix  : (K,K) 或 (batch_size,K,K)
    """
    def __init__(self, n_spin_orbitals: int, n_states: int = 2, hidden_dim: int = 16, *, rngs: nnx.Rngs):
        super().__init__()
        self.K = n_states
        self.n_spin = n_spin_orbitals

        # 初始化 K 个独立单态波函数
        self.single_ansatz_list = [
            SingleStateAnsatz(n_spin_orbitals, hidden_dim, rngs=rngs)
            for _ in range(self.K)
        ]

    def __call__(self, x: jax.Array):
        # -----
        # 单样本处理函数 (核心)
        # x_single: (K, n_spin)
        # -----
        #print(f'收到了x.shape={x.shape}')
        def _forward_single(x_single):
            # 构造 KxK 矩阵 M: M[i,j] =  $\psi_j(x_i)$ 
            #print(f'x_single.shape={x_single.shape}')
            M = []
            for i in range(self.K):
                row = []
                for j in range(self.K):
                    val = self.single_ansatz_list[j](x_single[i])
                    row.append(val)
                M.append(jnp.stack(row))
            M = jnp.stack(M) # (K, K)

```

```

        log_det = jnp.linalg.det(M)
        return log_det, M

# -----
# 自动判断: 单条 / 批量
# -----
if x.shape[-1] == self.n_spin:
    #print('A')
    log_psi, log_M = _forward_single(x)
else:
    #print('B')
    x = x.reshape(-1, K, self.n_spin)
    #print(f'转换后x.shape={x.shape}')
    log_psi, log_M = jax.vmap(_forward_single)(x)
    return log_psi, log_M
def create_machine(model: NESTotalAnsatz):
    """将 Flax NNX 模型包装为 NetKet 风格的 machine 函数"""
    graphdef, state = nnx.split(model)

    @jax.jit
    def machine(params, sigma):
        #print(f'x.shape: {sigma.shape} ')
        m = nnx.merge(graphdef, params)
        log_psi_total, log_M_matrix = m(sigma)
        return log_psi_total

    return machine, graphdef, state

```

在这里我要重点介绍一下维度问题: 由于 `hi_ext.all_states().shape = (N,K*n_spin_orbitals)` 比如当 $K=2$ 的时候, `shape = (16,8)` 但是 `NESTotalAnsatz` 应该接受一个 $(K,n_spin_orbitals)$ 或者 $(batch_size,K,n_spin_orbitals)$ 因此你可以看出来我在 `NESTotalAnsatz` 内部的 `__call__` 针对接受 `x` 的做出了 `reshape`。此外 `sampler` 产生的 `samples` 同样有维度问题 原生会产生的维度有两种: $(n_)$